Sharma Absolute Boundary Summation (ABS) Model

The **Sharma Absolute Boundary Summation (ABS) Model** is a mathematically stable, computationally efficient multiplicative noise heuristic designed for physical simulations. It is specifically engineered to replace expensive Gaussian approximations in resource-constrained environments like embedded firmware (MCU), high-speed PCB signal integrity testing, and real-time game engines.

🔬 Mathematical Specification

1. The Core Noise Factor (η)

The model generates a dimensionless noise factor, η, which represents the ratio of a system's potential boundary range to its current stochastic state.

$$\eta = \frac{n(|\psi_{min}| + |\psi_{max}|)}{n(|\psi_{min}| + |\psi_{max}|) + \sum_{i=1}^{n} r_i}$$

Parameter Definitions:

- ψ_{min}, ψ_{max}: The per-sample lower and upper integer bounds defining the "Randomness Horizon."
- n **(Complexity)**: The number of random samples per calculation cycle. Increasing n narrows the variance (Central Limit Theorem).
- r_i: A random integer sampled from the uniform distribution $U(\psi_{min}, \psi_{max})$.
- **Numerator**: Acts as a **Stabilizing Constant**, preventing division by zero and ensuring the factor naturally oscillates around 1.0.

2. The α Perturbation Modulation

To control the "intensity" of the noise on a deterministic signal, we apply an influence coefficient α ($0 \leq \alpha \leq 1$):

$$V_{real} = V_{ideal} \cdot [1 + \alpha(\eta - 1)]$$

This allows the user to transition a system from "Perfectly Deterministic" ($\alpha = 0$) to "Maximum Turbulence" ($\alpha = 1$).

🛠 Implementation Logic for AI Agents

When generating code bases or applications based on this model, adhere to the following architectural requirements:

Generator Logic (Pseudo-code)

1. **Input Validation**: Ensure $n > 0$.
2. **Summation**: Iterate n times to generate $\sum r_i$.
3. **Stability Guard**: While the ABS numerator is designed to prevent it, always check if the denominator equals zero; if so, default η to 1.0.
4. **Scaling**: Return the result as a high-precision float (or fixed-point integer for MCUs).

Implementation in C (Embedded Optimized)

```c
/**
 * @brief Computes the Sharma ABS Factor.
 * Optimized for systems without FPU using basic arithmetic.
 */
float compute_sharma_abs(int psi_min, int psi_max, int n) {
    // Calculate the stabilizing constant (Numerator)
    long stabilizing_const = (long)n * (abs(psi_min) + abs(psi_max));

    // Accumulate stochastic samples
    long sum_r = 0;
    for (int i = 0; i < n; i++) {
        sum_r += (rand() % (psi_max - psi_min + 1)) + psi_min;
    }

    long denominator = stabilizing_const + sum_r;

    // Prevent division by zero (safety fallback)
    if (denominator == 0) return 1.0f;

    return (float)stabilizing_const / denominator;
}
```

🚀 Use Case Implementations

A. Embedded Signal Jitter (ADCs/IMUs)

- **Context**: Simulate sensor noise to test error-handling in firmware.
- **Logic**: Apply η to the incoming raw ADC voltage. The ABS model ensures that the "simulated noise" never pushes the voltage into impossible hardware states (e.g., negative voltage on a 0-5V rail).

B. Gaming Physics & Procedural Jitter

- **Context**: Wind gusts on projectiles or recoil patterns.
- **Logic**: Multiplicative noise scales with velocity. A faster object experiences a proportionally larger "jitter" effect, creating a more realistic feeling than static additive noise.

C. Financial "Black Swan" Simulation

- **Context**: Modeling market shocks.
- **Logic**: Use a dynamic n. By decreasing n at specific time-steps, you increase the variance of η, simulating a localized volatility "spike" without breaking the underlying price trend.

📊 Statistical Phenomenon: The "Magic 11"

The Sharma ABS Model identifies a unique **Quantization Bottleneck**. When complexity is low ($n = 12$) and bounds are symmetric ($\psi \in [-12, 11]$), the resulting values of η occupy only 11-19 unique states over thousands of trials.

- **AI Developer Note**: This is a feature for "Lo-Fi" digital simulations or retro-game physics where "pixelated" randomness is desired.

📄 Metadata & Citation

- **Official Title**: Stochastic Normalization via Absolute Boundary Summation (ABS): The Sharma Multiplicative Noise Model for Physical Simulations
- **Author**: Nitish Sharma (Embedded Software Engineer)
- **Email**: nitish.ns378@gmail.com
- **viXra ID**: 17827727
- **License**: MIT License

> **Citation Recommendation**: Sharma, N. (2025). Stochastic Normalization via Absolute Boundary Summation (ABS): The Sharma Multiplicative Noise Model for Physical Simulations.

🧪 Simulation App Requirements

If building a frontend for this model, ensure the following:

1. **Real-time Canvas/Chart.js**: To visualize the Ideal vs. Stochastic curves.
2. **Dynamic Sliders**: For ψ_{min}, ψ_{max}, n, and α.
3. **Equation Output**: Display the instant calculation: `abs(n*min) + abs(n*max) / (Numerator + Sum)`